

Day 7 — Two's Complement Connection

Module: Nikhilam Subtraction · **Band:** Middle · **Time:** 45 min

Learning objective

By the end of this lesson, students will identify the structural similarity between Nikhilam (“all from 9, last from 10”) in base 10 and **two's complement** (“flip all bits, add 1”) in base 2, and will subtract two small binary numbers using the two's-complement method.

Materials

- Whiteboard
- Printed 8-bit binary number line poster (0000 0000 to 1111 1111)
- Pencil + ruled paper
- Optional: a calculator with a binary mode (most smartphone calculators have one)

Vocabulary introduced today

- *binary* — base 2 (the number system computers use).
- *bit* — a binary digit; either 0 or 1.
- *two's complement* — the binary version of the Nikhilam principle.

Lesson flow

Warm-up (5 min)

- Sūtra chant.
- Quick fire: 3 base-10 complements (to 100, 1000, 10 000).
- Provocation: “*Same trick. Different number system. Where else might this work?*”

Core (25 min) — From base 10 to base 2

1. The Nikhilam pattern, restated in base-neutral language:

To compute $b^n - A$:

- Each digit of A : subtract from $(b - 1)$ — the maximum digit in that base.
- The last digit: subtract from b .

In base 10: $(b - 1) = 9$ and $b = 10 \rightarrow$ “all from 9, last from 10.”

2. In base 2. “*The biggest digit in base 2 is 1. So ‘(b - 1)’ is 1. The base b is 2.*”

Pattern becomes:

- Each bit of A : subtract from 1. (This is the same as **flipping the bit**: $0 \rightarrow 1$, $1 \rightarrow 0$.)

- The last bit: subtract from 2. ($2 - 0 = 10$ in binary, $2 - 1 = 1$. Equivalent: “flip the last bit and add 1” — except we just add 1 to the whole flipped result and let the carry propagate.)

That’s exactly the **two’s complement** algorithm taught in computer science: > **To negate a binary number: flip all bits, then add 1.**

3. Worked example. Subtract $5 - 3$ using two’s complement, 4 bits:

- $5 = 0101$. $3 = 0011$.
- To compute $5 - 3$, we compute $5 + (-3)$, where -3 is the two’s complement of 3.
- Two’s complement of 3 (4 bits): flip all bits of $0011 \rightarrow 1100$, then add 1 $\rightarrow 1101$.
- Now add: $0101 + 1101 = 1\ 0010$.
- The leading 1 (the 5th bit) is the **overflow / carry out** — we discard it. (This carry-out is the “underflow” of the $16 = 2^4$ base, exactly like the $10\ 000$ in $10\ 000 - 4567$.)
- Result: $0010 = 2$. ✓

4. Why this is the SAME as Nikhilam.

- In base 10: $10\ 000 - 4567 = 5433$. Same calculation: complement of 4567 = 5433, and $4567 + 5433 = 10\ 000$.
- In base 2: $16 - 3 = 13 = 1101$. That’s the two’s complement of 3. And $0011 + 1101 = 1\ 0000 = 16$. ✓
- Then we subtract THAT result from 5 (or equivalently add it to 5 modulo 16): $5 - 3 \equiv 5 + (16 - 3) \pmod{16} = 5 + 13 = 18 \equiv 2 \pmod{16}$.

“Tirthaji’s six-word sūtra and the two’s-complement circuit in your phone’s CPU are doing the same algebra. The CPU just does it in base 2.”

- ### 5. Why computers use this.
- A CPU has no separate “subtract” circuit — it has an “add” circuit, an “invert bits” circuit, and a “+1” (increment) circuit. Two’s complement lets it do subtraction with just those. This is the deepest reason: the Nikhilam principle saves silicon.

Activity (10 min) — Try it

- Worksheet: 4 small subtractions to do via two’s complement (4 bits, all results positive).

Problem	A	B	Flip A or B?	Two’s-comp	Add	Discard top bit	Answer
$6 - 4$	0	0	Flip B	1100	1	drop top $\rightarrow$ 0010	2 ✓
	1	1			001		
	1	0			0		
	0	0					
$7 - 5$	0	0	Flip B	1011	1	drop top $\rightarrow$ 0010	2 ✓
	1	1			001		
	1	0			0		

Problem	A	B	Flip A or B?	Two's-comp	Add	Discard top bit	Answer
	1	1					
9 – 2	1	0	Flip B	1110	1	drop top → 0111	7 ✓
	0	0			011		
	0	1			1		
	1	0					
12 – 7	1	0	Flip B	1001	1	drop top → 0101	5 ✓
	1	1			010		
	0	1			1		
	0	1					

- Students work in pairs. One does the binary math; the other verifies with a calculator.

Wrap-up (5 min)

- “You’ve just done in your head what a CPU does in nanoseconds. Different scale, same logic.”
- Preview Day 8: “Tomorrow — what if the number we’re subtracting FROM isn’t a clean power of ten? E.g. 8243 – 567. Nikhilam still helps, but we need one more move.”

Student-facing content

Two’s Complement = Nikhilam in Binary

	Base 10 (Nikhilam)	Base 2 (Two’s Complement)
Maximum digit	9	1
“All from (max)”	All from 9	Flip each bit
“Last from base”	Last from 10	Add 1

Same algebra. Same trick. The CPU inside your phone uses this every time it computes a subtraction.

“*Nikhilam Navataścaramam Daśataḥ*” — Tirthaji’s 1965 sūtra. The base-2 cousin was independently discovered in 1945 by the team designing the EDVAC computer.

Homework

- Convert 13 – 5 to binary (4 bits) and solve via two’s complement. Show your work.
- One reflection sentence: “What surprised me about the binary connection.”

Differentiation

- Tier up:** Try **negative results**. 3 – 5 in 4 bits gives 1110, which is the two’s-complement representation of –2. Look up how CPUs read this.
- Tier down:** Skip the binary math. Just understand the structural parallel: “flip the digits, add 1, same idea.” Repeat the base-10 examples instead.

Teacher notes

- This is the most cross-disciplinary day of the module. Some students will be enthralled; others will find binary intimidating. Both reactions are fine.
- Many students have *heard* of binary; few have *done* arithmetic in it. The 4-bit examples are deliberately small.
- A natural question: “*Does this mean Tirthaji invented two’s complement?*” — No. Two’s complement was independently developed for digital computing in the 1940s. What’s beautiful is the **structural parallel** — the same algebraic principle $(b^n - A = (b^n - 1) - A + 1)$ plays the same role in both base 10 and base 2.

Citation(s) used in this lesson

- Tirthaji, *Vedic Mathematics* (1965), sūtra Nikhīlam (#2).
- Patterson & Hennessy, *Computer Organization and Design*, two’s-complement section — standard CS reference for the binary parallel.